Section 3.2: Gradient Descent

How Machines Learn by Walking Downhill

AI Class — Lecture Notes [Student Copy]

---

1 Why We Need a New Strategy

In Section 1.1 (linear regression) we minimised the MSE loss by setting $\partial J/\partial w = 0$ and solving analytically. This gave a clean, exact answer in one step — the *normal equations*.

But that only works because the loss was a simple convex quadratic in w. For a neural network the loss depends on weights through several layers of non-linear activation functions. Setting all partial derivatives to zero produces a system of equations with **no closed-form solution**.

We therefore need an **iterative** approach:

1. Start with some initial guess for the weights.
2. Compute how the loss changes with each weight (the gradient).
3. Nudge every weight in the direction that reduces the loss.
4. Repeat until the loss stops improving.

This procedure is called **gradient descent**, and it is the foundation of virtually all modern machine-learning optimisers.

2 The Gradient as a Compass

Recall that for a function $J(w)$, the derivative dJ/dw measures the *slope* of J at the current value of w:

Sign of dJ/dw	Meaning
> 0	J rises as w increases $\Rightarrow$ **decrease** w
< 0	J falls as w increases $\Rightarrow$ **increase** w
$= 0$	J is flat here $\Rightarrow$ we are at a (local) minimum

Both non-zero cases are captured by a single rule: subtract a small multiple of the gradient from the current weight.

Gradient Descent Update Rule

$$w \longleftarrow w - \eta \frac{dJ}{dw}$$

where $\eta > 0$ (pronounced "eta") is the **learning rate**: a small positive constant controlling the step size.

Applying this update repeatedly is called **gradient descent** because we are literally descending the loss surface step by step.

3 A Hand-Calculated Example

Example 3.1. Setup. One training point: $x = 2$, $y = 4$. Model (no bias): $\hat{y} = wx$. Loss:

$$J(w) = (y - wx)^2 = (4 - 2w)^2.$$

The true weight is clearly $w^* = 2$, since $\hat{y} = 2 \times 2 = 4 = y$. We start at $w_0 = 0$ and run gradient descent with learning rate $\eta = 0.1$.

3.1 Deriving the Gradient

Differentiate $J(w) = (4 - 2w)^2$ with respect to w using the chain rule:

$$\frac{dJ}{dw} = 2\,(4 - 2w) \cdot (-2) = \underline{\qquad\qquad\qquad\qquad}$$

At $w = 0$:

$$\left.\frac{dJ}{dw}\right|_{w=0} = -4\,(4 - 0) = \underline{\qquad\qquad}$$

The gradient is *negative*, telling us that J decreases as w increases — so the update will increase w. That makes sense: we need to move from $w = 0$ towards $w^* = 2$.

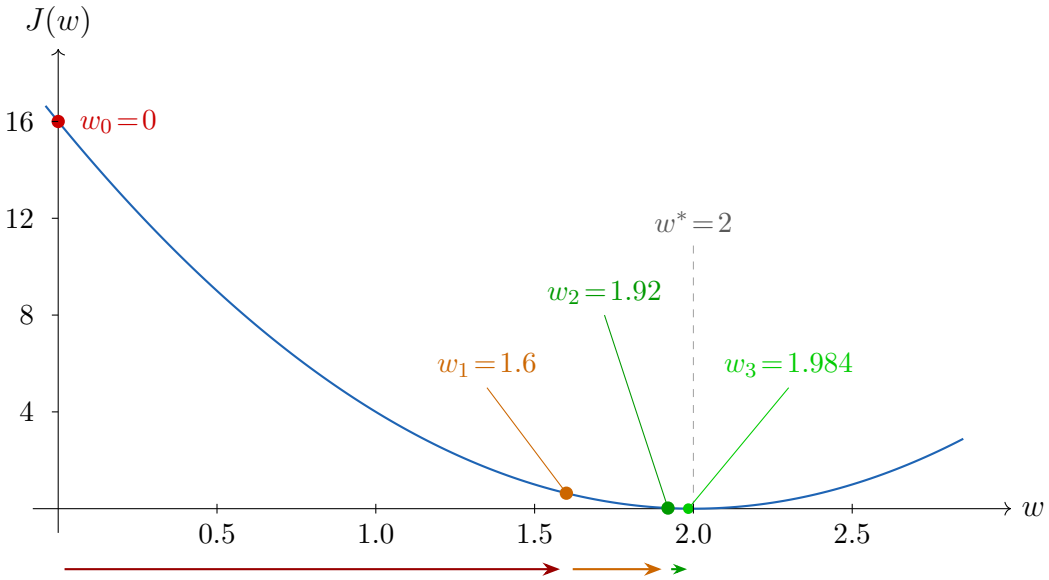
3.2 Four Update Steps

Apply $w \leftarrow w - 0.1\,\dfrac{dJ}{dw}$ repeatedly, using $\dfrac{dJ}{dw} = -4(4 - 2w)$ each time:

Step t	w_t	$J(w_t)$	dJ/dw at w_t	$w_{t+1} = w_t - 0.1 dJ/dw$
0	0	_____	_____	_____
1	1.6	_____	_____	_____
2	1.92	_____	_____	_____
3	1.984	_____	_____	_____

After only four steps from $w_0 = 0$, the weight has reached 1.997 — within 0.15% of the true value $w^* = 2$. Note that the gradient shrinks at each step, so the updates automatically become smaller as we approach the minimum.

3.3 Visualising the Descent



The dots show the position of w_t on the loss curve at each step. Each step is smaller than the last because the gradient — and hence the slope of the curve — becomes flatter as we near the bottom. Also check out the interactive visualization at LINK.

4 The Learning Rate η

The learning rate η controls how large each step is. Choosing it well is important: too small and training is painfully slow; too large and the updates overshoot the minimum.

Too Small: $\eta = 0.01$

Using the same example, first update:

$$w_1 = 0 - 0.01 \times (-16) = _____$$

The algorithm will still converge to $w^* = 2$, but it now needs roughly ten times as many steps.

Too Large: $\eta = 0.5$

First update:

$$w_1 = 0 - 0.5 \times (-16) = _____$$

The weight *overshoots* past $w^* = 2$ all the way to 8. Check what happens next:

$$\left.\frac{dJ}{dw}\right|_{w=8} = -4(4 - 16) = _____ \qquad \Rightarrow \qquad w_2 = 8 - 0.5 \times 48 = _____$$

The loss is now *larger* than it was at $w_0 = 0$, and the updates will keep growing. The algorithm **diverges**.

Symptom of a bad learning rate: if J increases during training, η is almost certainly too large. A safe starting point for most problems is $\eta = 0.01$.

5 Visualing Gradient Descent in 3D

If we have two weights w_1 and w_2, the loss surface is a 3D landscape. The gradient is a vector pointing in the direction of steepest ascent.

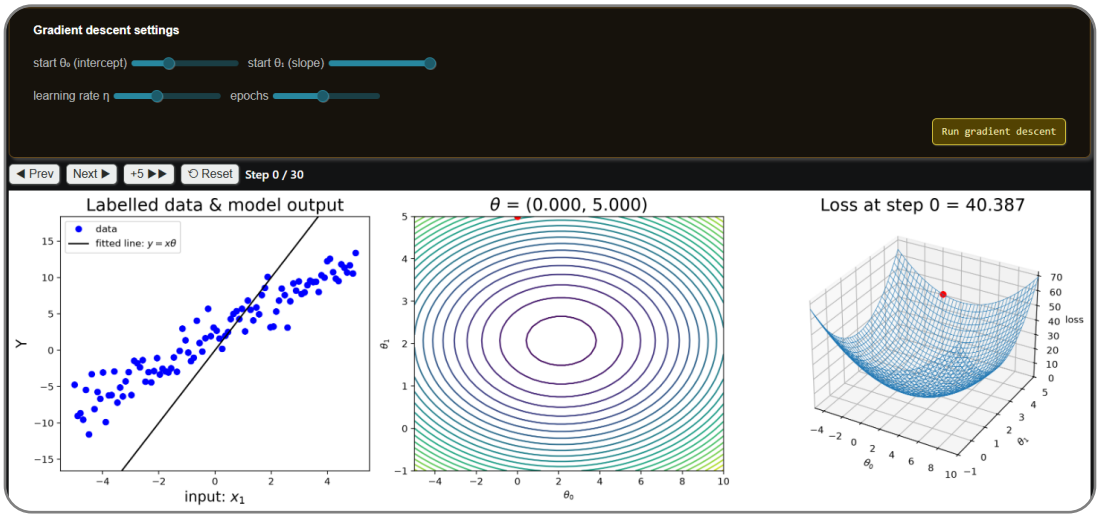


Figure 1: Visualizing Gradient Descent in 3D

6 From One Weight to a Neural Network

Everything above applies to a network with many weights. Suppose the network has weight matrices $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}$ and biases $\mathbf{b}^{(1)}, \mathbf{b}^{(2)}$. After every forward pass we apply:

$$W_{ij}^{(\ell)} \longleftarrow W_{ij}^{(\ell)} - \eta \frac{\partial J}{\partial W_{ij}^{(\ell)}} \qquad \text{for every layer } \ell \text{ and every entry } (i, j).$$

The structure is identical to the 1-weight example — only the number of updates differs.

The sole new challenge is *computing* $\partial J / \partial W_{ij}^{(\ell)}$ for each weight. Because the loss depends on a weight only indirectly — through several layers of non-linear activation functions — these gradients are not obvious to write down.

Coming next. In Section 3.3 we build a concrete two-layer network and trace a prediction step by step (the *forward pass*). In Section 3.4 we derive the gradients by working backwards through the same network (the *backward pass*, or **backpropagation**). Only then will we have everything needed to run gradient descent on a neural network.